Streaming Self-Improvement of Language Models via Online Self-Distillation from Verifiable Feedback

Anonymous authors
Paper under double-blind review

Abstract

Deployed language-model agents face a stream of tasks and receive only sparse, verifiable feedback—a unit test passes, an answer matches, a query executes. We study how a model can permanently improve from this signal alone, online, without ground-truth labels or a stronger teacher. We propose an online self-distillation method: each incoming problem is answered and scored before any update; verified successes enter a memory that serves both retrieval-based in-context learning and, crucially, self-distillation—the model conditioned on its own verified answer acts as teacher for the same model without it, distilling the answer's information into the weights. A forward-looking re-evaluation pass lets the improving model recover previously failed problems, converting weight gains back into training data. On seven diverse StreamBench tasks (code, SQL, diagnosis, QA, tool use), our method beats zero-shot, streaming-ICL, and REINFORCE++ baselines on every task. Under distribution shift—three domains interleaved into one stream—a single online-distilled model matches or beats three per-domain specialists, with positive cross-domain transfer to code (+3.4 points), while retrieval self-segregates (99.9% same-domain), showing the transfer lives in the weights; the RL baseline collapses on the same stream. Finally, comparing online training to multi-epoch batch self-distillation on identical data, the online-trained model masters the source data better (+5.5 points) [TODO: and transfers better to an unseen domain (results pending)]. Together these results position online self-distillation as a simple, stable recipe for continual self-improvement from verifiable feedback.

1 Introduction

Language models deployed as agents encounter their tasks as a stream: one problem at a time, in an order they do not control, with feedback that is sparse but verifiable—a unit test passes, a SQL query returns the right rows, a diagnosis matches the record. Two families of methods exploit this signal today. Memory-based approaches such as streaming in-context learning retrieve past verified successes as few-shot demonstrations; they adapt instantly but the model itself never improves, and gains vanish when memory is unavailable. Reinforcement learning from verifiable rewards updates the weights, but a scalar reward per attempt is a thin learning signal, and—as we show—policy optimization can collapse catastrophically on long heterogeneous streams.

We study a third route: online self-distillation. When the model solves a streamed problem and the oracle verifies it, the model conditioned on its own verified answer becomes a teacher for the same model without that answer. Matching the teacher's hint-informed next-token distribution moves the answer's information into the weights—a dense, token-level signal derived entirely from the model's own verified output. No gold labels, no external teacher, no preference pairs.

Our method (§3) couples this distillation step with a reward-filtered memory used for kNN in-context retrieval, and a forward-looking re-evaluation pass: problems the model failed on arrival are periodically retried with the current, improved model; late successes enter memory and become training signal. This closes a flywheel—weight gains produce new

Table 1: Method taxonomy: what each method updates and what signal it consumes. Online SDFT is the only method that both updates weights and exploits a dense, token-level training signal derived from verified self-generated text.

Method	Updates M	Updates θ	Training signal	Signal density
Zero-shot (Base)	–	–	–	–
Self-StreamICL	✓	–	– (retrieval only)	–
REINFORCE++	–	✓	scalar reward	per sequence
Online SDFT (ours)	✓	✓	verified self-output	per token

verified data, which produces further weight gains—that pure memory methods and pure RL both lack. Scoring always precedes any use of a problem for training, so the streaming metric is strictly predict-then-train.

Contributions.

- A simple online self-distillation recipe for streaming self-improvement from verifiable 0/1 feedback only (§3).
- State-of-the-art streaming accuracy on all seven StreamBench tasks spanning code, SQL, medical diagnosis, multi-hop QA, and tool use, against zero-shot, streaming-ICL, and REINFORCE++ baselines (§4.2).
- Robustness to distribution shift: on a stream interleaving three domains, one shared model matches or beats three per-domain specialists, with positive cross-domain transfer to code; retrieval self-segregates by domain (99.9%), so the transfer is carried by the weights. The RL baseline collapses on the same stream (§4.3).
- Online beats batch: trained on identical problems with the same loss and model, the online-trained model masters the source data better than multi-epoch batch self-distillation (+5.5 points) [TODO: and transfers better to an unseen target stream] (§4.4).

2 Problem Formulation

Streaming setting. Following the online evaluation protocol of StreamBench (Wu et al., 2024), an agent faces an input–feedback sequence $(x_1, fb_1), (x_2, fb_2), \ldots, (x_T, fb_T)$ drawn from a task distribution and presented in a fixed, seeded order. At step t the agent produces

$$\hat{y}_t \;=\; f\big(p(x_t,\, r(M_t)) \,\big|\, \theta_t\big), \tag{1}$$

where θ_t are the current model weights, M_t is an external memory, $r(\cdot)$ a retriever, and $p(\cdot)$ a prompting template. The environment returns binary verifiable feedback $fb_t = g(x_t, \hat{y}_t) \in \{0, 1\}$—execution of generated code against unit tests, execution-match of SQL, exact/label match for QA and diagnosis. Gold references exist only inside g; the agent observes the bit. After feedback, the agent may update any of (p, r, M, θ); the methods we study differ precisely in which of these they update (Table 1).

Evaluation. Performance is prequential: $\hat{y}_t$ is produced and scored before x_t influences any component, and the stream is consumed once. We report final cumulative accuracy $\mathrm{Acc}_T = \frac{1}{T}\sum_{t=1}^{T} h(\hat{y}_t, y_t)$ (each task's native metric h) and, equivalently, cumulative regret $R_T = \sum_{t=1}^{T}(1 - h(\hat{y}_t, y_t))$, which penalizes late learning. All methods on a given task see the identical stream order.

Why this setting is hard. The agent gets one attempt per problem, a single bit per attempt, no gold text ever, and no second pass over the stream. Improvements must therefore be bootstrapped from the agent's own verified outputs—and any weight update risks destabilizing performance on the remainder of the stream, a risk realized dramatically by the RL baseline in §4.3.

3 Method: Online Self-Distillation from Verifiable Feedback

Setting. Problems $x_1, x_2, \ldots$ arrive in a fixed stream in batches of B=10. For each problem the environment provides a binary oracle $r(x, y) \in \{0, 1\}$ (test execution, exact match, label equality). Gold answers exist only inside the oracle; only the bit leaves it. The streaming metric is prequential: every problem is answered and scored before any update that involves it. We maintain a policy π_θ (base model + LoRA), a frozen copy of the base model π_{ref}, and a memory $M : x \mapsto (y^\star, e)$ storing each problem's latest verified answer and an embedding of its question.

Per-batch loop. Each stream batch triggers three phases (Algorithm 1):

(1) Score. For each new x: retrieve the k=3 nearest past problems from M (embedding kNN, excluding x) as in-context demonstrations, generate greedily, and query the oracle. The result is recorded—permanently—and, if correct, the answer is stored in M (reward filter).

(2) Forward re-evaluation. Problems from the last W=9 batches are re-answered with the current model and re-scored. Successes update M: failures recovered by the improved model enter memory late, and existing entries are refreshed into the current model's phrasing (keeping distillation targets near on-policy). Re-evaluation never touches recorded scores.

(3) Self-distillation. For each window problem x with $M(x) = y^\star$: the student sees prompt $P(x)$ = system + demos + x; the teacher sees $P(x)$ followed by $y^\star$ as an assistant turn and x repeated. A completion $\hat{y}$ is generated from the teacher prefix, and we minimize the token-level forward KL

$$\mathcal{L}(\theta) \;=\; \sum_t \mathrm{KL}\big(\pi_{\text{ref}}(\cdot \mid P^+(x), \hat{y}_{<t}) \;\big\|\; \pi_\theta(\cdot \mid P(x), \hat{y}_{<t})\big)\,, \tag{2}$$

where P^+ denotes the hint-augmented prompt. The teacher is the frozen base model—the only asymmetry is the hint. The student learns to produce the teacher's hint-informed distribution without the hint, compressing the verified answer into the weights. We use 2 epochs per window pool, LoRA (r=16, α=32, all linear layers), learning rate $5{\times}10^{-5}$, and skip the first 3 completion tokens.

Why no leakage. The hint is always the model's own earlier oracle-passing output, never a dataset field; demos always come from strictly earlier batches; and a problem's score is final before it can enter M. We audit these causal-ordering invariants programmatically over the actual run logs (Appendix A).

Generation source. Conditioning the generated completion $\hat{y}$ on the hint (vs. sampling it from the bare student prompt, as in offline self-distillation) turns out to be a minor, bootstrap-phase choice: the two settings differ by $\sim$1 point and only early in the stream (§4.5), so our results do not hinge on it.

4 Experiments

4.1 Setup

Tasks. We evaluate on seven StreamBench tasks spanning five domains and four oracle types (Table 2). LiveCodeBench (Jain et al., 2025): competitive-programming problems; the oracle executes generated code against hidden unit tests. DS-1000 (Lai et al., 2023): StackOverflow-derived data-science problems over seven Python libraries; the oracle executes the completion inside each problem's test harness (we exclude TensorFlow problems, which conflict with our training stack, leaving 955). Spider (Yu et al., 2018) and BIRD (Li et al., 2023): text-to-SQL with execution-accuracy oracles against live databases, BIRD adding large, noisy, domain-specific schemas. DDXPlus (Fansi Tchango et al., 2022): differential diagnosis from synthetic patient profiles, 49-way classification with label-match oracle. HotpotQA (Yang et al., 2018): multi-hop QA in the distractor setting, exact-match

Algorithm 1 Online SDFT (one stream batch)

```
for each new problem x in batch B_t do
    y ← π_θ( system + kNN_3(M, x) + x )                    (greedy)
    record r(x, y)                                          (prequential metric; final)
    if r(x, y) = 1 then
        M[x] ← y
    end if
end for
for each x in last W batches do
    y' ← π_θ( system + kNN_3(M, x) + x )
    if r(x, y') = 1 then
        M[x] ← y'                                           (recover / refresh)
    end if
end for
for each x in window pool with M[x] = y* do
    ŷ ~ π_θ(P+(x));   update θ by Eq. 2
end for
```

Table 2: Streamed tasks: size T, native metric h, and oracle g.

Task	Domain	T	Metric	Oracle g
LiveCodeBench	competitive code	1055	avg reward	unit-test execution
DS-1000	data-science code	955	pass@1	test-harness execution
Spider	text-to-SQL	2147	execution acc.	SQL execution match
BIRD	hard text-to-SQL	1534	execution acc.	SQL execution match
DDXPlus	medical diagnosis	1764	accuracy	label match
HotpotQA	multi-hop QA	1500	exact match	span match
ToolBench	function calling	750	strict EM	name+args match (+judge)

oracle. ToolBench (STE-curated subset; Wang et al., 2024): single-turn function calling; strict oracle on API name and arguments, with an LLM judge for free-form argument fields. Stream orders are fixed (seed 42) and identical across methods, following StreamBench's released-sequence protocol.

Baselines. Base: zero-shot with the task's system prompt; no memory, no updates; the floor any adaptive method must beat. Self-StreamICL (Wu et al., 2024): the strongest streaming baseline from StreamBench. Outputs verified correct (fb_t=1) are stored as $(x, \hat{y})$ pairs; at each step the k=3 nearest stored pairs (cosine similarity of question embeddings) are prepended as demonstrations. Weights are frozen; StreamBench showed that storing only correct self-outputs is critical, and our method inherits exactly this filter. REINFORCE++ (Hu, 2025): a policy-gradient baseline under the identical rolling-window skeleton as our method: the greedy answer to each new problem is scored once (the same oracle call that produces the metric), stored, and replayed for training over the window with batch-mean advantages, global advantage whitening, PPO-style clipping, and a KL penalty (β=0.01) to the frozen base policy. It is the "update θ from the scalar bit" counterpart of our method.

Implementation. All methods use Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) with greedy decoding; our method and REINFORCE++ train LoRA adapters (Hu et al., 2022) (r=16, α=32, all linear layers) with AdamW. Retrieval uses Qwen3-Embedding-0.6B. Stream batch size 10, window W=9; distillation runs 2 epochs per window pool at learning rate 5×10^{-5}. Each run fits on a single H200 GPU. Full hyperparameters in Appendix B; oracle-call accounting in §4.5.

4.2 Seven streaming benchmarks

Table 3 reports final cumulative accuracy over each full stream. Our method attains the best accuracy on all seven tasks. Three patterns stand out.

Table 3: Final cumulative accuracy on seven StreamBench tasks (native metric per task; identical stream order and base model for all methods). Best in bold; our method is best on all seven.

Benchmark	Domain	Size	Base	ICL	R++	Ours
LiveCodeBench	code	1055	0.338	0.424	0.386	0.448
DS-1000	Python	955	0.325	0.367	0.369	0.423
Spider	text-to-SQL	2147	0.747	0.778	0.587	0.808
DDXPlus	medical dx	1764	0.436	0.647	0.404	0.664
BIRD	hard SQL	1534	0.330	0.346	0.271	0.354
HotpotQA	multi-hop QA	1500	0.521	0.509	0.537	0.562
ToolBench	func-calling	750	0.549	0.593	0.607	0.615

Table 4: Fused-stream results (4,219 problems). "SDFT standalone" aggregates the three per-domain runs at matched prefixes (regret summed, accuracy weighted). One online-distilled model matches or beats three specialists; REINFORCE++ collapses.

Method	Acc	Cum. regret	DS-1000	DDXPlus	HotpotQA
Base	0.439	2365	0.318	0.436	0.521
ICL k=3	0.533	1971	0.358	0.647	0.509
REINFORCE++ (fused)	0.253	3153	0.213	0.262	0.267
SDFT standalone (3×)	0.573	1800	0.423	0.664	0.562
SDFT fused (ours)	0.585	1750	0.457	0.676	0.560

(i) Weight updates help precisely where retrieval fails. ICL is strong where neighbors are informative (DDXPlus: .647 vs base .436) but hurts where retrieved demonstrations mislead (HotpotQA: .509 vs base .521). Our method inherits ICL's retrieval yet still wins on HotpotQA (.562): the distillation channel injects only the model's own verified answer for the same problem—a signal that cannot mislead—and consolidates it into weights.

(ii) Gains are largest where the base model is weakest. DS-1000 (+9.8 points over base), LiveCodeBench (+11.0), DDXPlus (+22.8): tasks with low base accuracy leave the most headroom for bootstrapped supervision. On near-saturated Spider (.747 base) the method still adds +6.1.

(iii) Scalar-reward RL is inconsistent. REINFORCE++ improves modestly on some tasks (HotpotQA .537, ToolBench .607) but collapses below base on others (Spider .587 vs .747; DDXPlus .404 vs .436)—the thin per-sequence signal is unreliable across task types, foreshadowing §4.3.

4.3 Distribution shift: one model, three domains

Deployed agents rarely see a single homogeneous task. We construct a 4,219-problem stream interleaving DS-1000 (code), DDXPlus (diagnosis), and HotpotQA (QA)—three domains and three oracle types—by a seeded random merge that preserves each domain's internal order. Preserving within-domain order makes every per-domain slice of the fused run exactly comparable, problem-for-problem, to the standalone runs of §4.2: at the n-th code problem, the fused model has seen the same n code problems as the standalone model, plus $\sim 2n$ problems from other domains. Any per-domain difference therefore measures cross-domain transfer or interference, cleanly.

Table 4 and Figure 1 give the results.

(i) One model $\geq$ three specialists. The fused model beats the aggregated standalone runs overall (.585 vs .573) and is no worse on any domain, with positive transfer to the hardest domain (DS-1000: .457 fused vs .423 standalone, +3.4 points). Mixed-domain training on verified self-generated data helps rather than interferes.

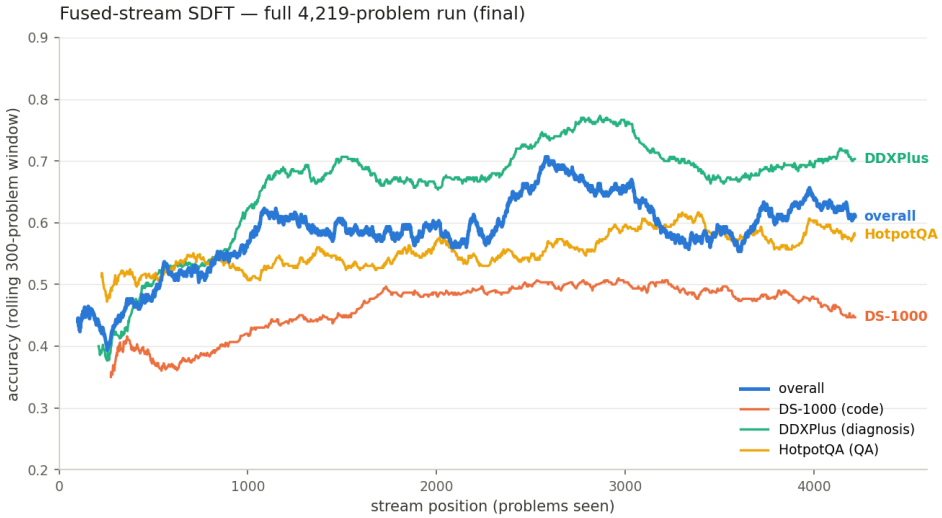

Figure 1: Rolling accuracy (300-problem window) of online SDFT over the fused three-domain stream, overall and per domain. All domains improve simultaneously; no interference from distribution shift.

(ii) Transfer is carried by the weights, not the prompts. One might suspect cross-domain demonstrations explain the gains. They cannot: retrieval self-segregates. Across 2,059 memory queries, 99.9% of retrieved demonstrations are same-domain (2 queries retrieve any cross-domain example) —question embeddings cluster by domain, so prompts are effectively domain-pure and near-identical between fused and standalone runs. The +3.4-point transfer must flow through the shared LoRA parameters.

(iii) The RL baseline collapses. REINFORCE++ tracks ∼5 points below our method for the first 2,000 problems, then diverges: approximate KL flips negative, PPO clip fraction spikes from 0.08 to 0.68, advantage standard deviation triples, and accuracy falls to zero for the remaining half of the stream (per-500 window accuracy .55 → .10 → .00; final .253). The identical hyperparameters are stable on every shorter standalone stream (§4.2), implicating long-horizon mixed-domain streams specifically. Our method's rolling accuracy instead climbs monotonically (.44 → .68 peak) with every domain improving simultaneously (Figure 1); across ∼12,000 streamed problems in this paper it never regressed.

4.4 Online vs. batch training

Is the online schedule itself load-bearing, beyond the loss? We compare the online DS-1000 run against batch self-distillation: the identical trainer, loss, base model, and oracle, run as classic offline epochs—four passes over the full 955-problem set; each pass generates an answer per problem with the current model, oracle-filters, and distills on the verified answers. No memory, no stream, no re-evaluation. We then measure source mastery: a single bare-prompt greedy pass over all 955 problems (no demonstrations, no memory) with each resulting adapter.

Table 5: Source-data mastery after training on DS-1000 (bare-prompt greedy pass, all 955 problems).

Model	Acc	Solved / 955
Base	0.313	313
Batch SDFT (4 epochs)	0.425	406
Online SDFT (ours)	0.480	458

Both regimes learn substantially (Table 5), but online wins by +5.5 points on the very data batch trained on for four epochs. The advantage is breadth, not overlap: online uniquely solves 123 problems vs 71 for batch (335 shared, 426 by neither), while forgetting relative to base is equal and small (37 vs 32 problems lost). We attribute the gap to the stream's implicit curriculum: each problem is revisited across a rolling window while the model evolves, and forward re-evaluation keeps converting newly in-reach problems into training signal—whereas single-sample batch generation can only train on what the current model already solves greedily. [TODO: Batch with n=5 sampled attempts per problem (exploration-matched, oracle-budget-matched at 2 epochs) — eval running.] [TODO: Transfer: both adapters warm-start online SDFT on the unseen HotpotQA stream; compare accuracy/regret vs from-scratch (.562/657). Runs pending.]

4.5 Ablations and cost

Generation source. Offline SDFT (Yang et al., 2024) samples the distillation completion from the bare prompt; we condition it on the hint. Flipping our setting to the offline default changes DS-1000 by +1.3 points in favor of the default, with the difference confined to the early stream; on the fused stream the two settings are even after a small early deficit (−10 regret in the first 300 problems, flat thereafter). The choice is a bootstrap-phase detail: hint-conditioning helps while the adapter is weak, bare-prompt generation is marginally better once it is strong. Our headline results use the (slightly disadvantaged) hint-conditioned setting.

Training pressure. Four distillation epochs per window beat two on DS-1000 (−21 cumulative regret at matched settings); doubling the learning rate instead does not recover the gap (+5 regret)—repetition, not step size, is what converts verified answers into weights.

Multi-sample distillation. Distilling from N=10 sampled candidates per problem—with or without similarity filtering against the hint—matches single-sample distillation on DS-1000 (a wash at matched oracle budget). Sample count is not the bottleneck; which problems yield a verified answer is.

Oracle budget. Our method issues ∼9.8 oracle calls per problem (1 scored arrival call + ∼8.8 re-evaluation calls that only gate memory); all baselines use exactly 1. The asymmetry does not explain the gaps: (i) re-evaluation with a frozen model recovers almost nothing—the extra calls are only useful because the model is improving; (ii) REINFORCE++ already shows that weight updates from 1 call/problem underperform; and (iii) in the online-vs-batch comparison the batch arm receives 4 calls/problem and still trails by 5.5 points. Full per-run call and GPU-hour accounting in Appendix C.

5 Analysis

We organize the analysis around four mechanism questions.

Q1: Where do the gains live—prompts or weights? Two independent probes give the same answer. First, in the fused experiment, retrieval self-segregates by domain (99.9% same-domain demonstrations), yet the fused model still transfers across domains (+3.4 on code)—prompts cannot carry that signal, weights must. Second, the source-mastery evaluation strips all scaffolding (no demonstrations, no memory) and the trained adapter retains a +16.7-point gain over base (.480 vs .313): the improvement survives removal of the entire prompting apparatus. ICL's gains, by construction, do not.

Q2: What does forward re-evaluation buy? Re-evaluation converts weight improvements back into training data. In the fused run, 84 of 1,463 memory entries (∼6%) came from problems failed on arrival and recovered later by the improved model—examples a frozen-model method could never harvest, since re-answering with an unchanged model reproduces the same failure. Recovery concentrates early (most recoverable problems are caught within

their window), after which re-evaluation's remaining role is refreshing stored answers into the current model's phrasing, keeping distillation targets near on-policy. This is the flywheel: updates improve the model; the improved model mints new verified data; new data drives updates.

Q3: Why is self-distillation stable where RL is not? The collapse signature of REINFORCE++ on the fused stream (negative approximate KL, clip-fraction spike, exploding advantage variance) is characteristic of policy-gradient divergence: the objective chases a moving, high-variance advantage estimate. The distillation objective (Eq. 2) has none of these moving parts—its target distribution comes from a frozen teacher, its targets are oracle-verified text, and disagreement between student and teacher is bounded by the hint's information content. Empirically, per-batch KL loss stays in $[10^{-3}, 10^{-1}]$ across all runs, and no run among ∼12,000 streamed problems ever regressed catastrophically. This echoes, in the online setting, the offline finding that self-generated targets minimize distributional drift (Yang et al., 2024): our stored answers are by construction near the model's own distribution, so each update is a small, verified step.

Q4: Why does the online schedule beat batch epochs on identical data? Three ingredients of the stream are absent in batch training. Curriculum: a problem enters training only when the model (current, not initial) solves it, so supervision always sits at the model's frontier. Repetition with change: a problem is revisited $\sim W$ times across its window while the model evolves, each pass distilling against a slightly different student. Late harvest: re-evaluation adds problems as they come in reach. Batch generation with a single greedy sample per epoch can only ever train on what the current model already solves deterministically; its coverage grew from 38.5% to 41.8% across four epochs while the online run ended with verified answers for 53.5% of problems. The mastery gap (.480 vs .425) tracks this coverage gap. [TODO: The n=5 exploration-matched batch arm tests whether sampling closes the coverage gap—and whether closing coverage also closes mastery and transfer.]

Integrity of the streaming metric. Because every reported number is produced by a model that had never seen that problem—scored before storage, demonstrations strictly from the past, hints strictly self-generated—the prequential metric doubles as a continual generalization measure. We release a causal-ordering audit that verifies these invariants over the raw run logs (all checks pass; Appendix A).

6 Related Work

Streaming adaptation and memory-based improvement. StreamBench (Wu et al., 2024) formalizes online LLM improvement under prequential evaluation with binary feedback and shows that Self-StreamICL—storing only verified-correct self-outputs and retrieving them as demonstrations—is the strongest single-agent streaming strategy, with the correctness filter being essential (storing incorrect outputs, even labeled as such, does not help). Our ICL baseline is exactly this method, and our memory inherits its filter. MemPrompt (Madaan et al., 2022) pioneered deployment-time memory of user feedback for prompt editing; reflection-style agents (Shinn et al., 2023) accumulate verbal self-critiques. All of these adapt the context; the model itself never improves, gains are bounded by the frozen model's ability, and they vanish without the memory. Our mastery evaluation (§4.4) makes this limitation concrete, and our method converts the same filtered memory into permanent weight gains.

Self-distillation and distillation from self-generated data. Offline SDFT (Yang et al., 2024) shows that fine-tuning on the model's own rewrites of gold answers—rather than the gold text itself—preserves general capabilities by keeping training targets inside the model's output distribution; their mix-ratio analysis ties forgetting causally to target-distribution shift. Generalized knowledge distillation (Agarwal et al., 2024) distills on-policy student samples against a teacher's distribution. SDPO (Hübotter et al., 2026) makes the model conditioned on environment feedback a self-teacher inside policy optimization, showing the resulting gradient is a policy gradient with dense logit-level advantages, with strong results

on rollout-batch RL for code. Our method transplants the self-teacher idea to the streaming setting: the conditioning signal is a verified answer harvested online from the model's own past, the loss is anchored to a frozen reference (no moving teacher), and the surrounding machinery (reward-filtered memory, forward re-evaluation) exists precisely to keep minting such answers from a single pass over the data. Where SDPO and GKD are offline/rollout-batch, we show the online schedule is itself a source of gains (§4.4).

Reinforcement learning from verifiable rewards. GRPO (Shao et al., 2024), RLOO (Ahmadian et al., 2024), and REINFORCE++ (Hu, 2025) optimize scalar verifiable rewards and power modern reasoning models. The scalar signal is thin—one bit per attempt—and credit assignment is per-sequence. Our experiments show a faithful REINFORCE++ under the identical streaming skeleton is unstable on long heterogeneous streams (§4.3); self-distillation extracts a per-token signal from the same oracle bit and remains stable.

Continual learning and test-time training. Our setting is continual learning without task boundaries (Parisi et al., 2019), sharing its central tension between plasticity and stability, but with verifiable feedback in place of supervised labels; the fused experiment is a task-incremental stress test in which our method exhibits transfer rather than interference. Test-time training (Sun et al., 2020) adapts per-instance and discards the update; our updates are cumulative and permanent. Self-improvement via self-training on filtered generations (Zelikman et al., 2022; Gulcehre et al., 2023) is the offline, multi-epoch ancestor of our per-window updates—our online-vs-batch comparison (§4.4) isolates exactly what the streaming schedule adds over that recipe.

7 Conclusion

We presented an online self-distillation method that turns sparse verifiable feedback into permanent model improvement: verified self-generated answers serve as hints for a self-teacher, a reward-filtered memory scaffolds retrieval and training, and forward re-evaluation converts weight gains back into data. The method wins on all seven StreamBench tasks, absorbs a three-domain distribution shift that collapses an RL baseline—with one shared model beating per-domain specialists via weight-borne transfer—and outperforms batch training with the identical loss on identical data. [TODO: Pending: online-vs-batch transfer to an unseen domain; batch with multi-sample exploration.] Limitations include the extra oracle calls spent on re-evaluation, single-model experiments at 7B scale, and binary-feedback tasks; richer textual feedback and larger scales are natural next steps.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. ICLR, 2024.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. ACL, 2024.

Arsene Fansi Tchango, Rishab Goel, Zhi Wen, Julien Martel, and Joumana Ghosn. DDXPlus: A new dataset for automatic medical diagnosis. NeurIPS Datasets and Benchmarks, 2022.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. arXiv preprint arXiv:2308.08998, 2023.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In ICLR, 2022.

Jian Hu. REINFORCE++: A simple and efficient approach for aligning large language models. arXiv preprint arXiv:2501.03262, 2025.

Jonas Hübotter et al. Reinforcement learning via self-distillation (SDPO). arXiv preprint arXiv:2601.20802, 2026.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. arXiv preprint arXiv:2409.12186, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. ICLR, 2025.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In ICML, 2023.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. NeurIPS, 2023.

Aman Madaan, Niket Tandon, Peter Clark, and Yiming Yang. Memory-assisted prompt editing to improve GPT-3 after deployment. EMNLP, 2022.

German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. Neural Networks, 2019.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. NeurIPS, 2023.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In ICML, 2020.

Boshi Wang, Hao Fang, Jason Eisner, Benjamin Van Durme, and Yu Su. LLMs in the imaginarium: Tool learning through simulated trial and error. ACL, 2024.

Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. Streambench: Towards benchmarking continuous improvement of language agents. In NeurIPS Datasets and Benchmarks, 2024.

Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. ACL, 2024.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In EMNLP, 2018.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In EMNLP, 2018.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. STaR: Bootstrapping reasoning with reasoning. NeurIPS, 2022.

A Leakage Audit

We verify four causal-ordering invariants programmatically over the raw run logs of the fused experiment (audit script released with the code): (A) every problem is scored exactly once (after de-duplicating checkpoint-resume rows); (B) no memory entry predates its problem's scoring batch—the scored generation can never see its own hint; (C) memory is append-only with respect to the stream, so retrieved demonstrations always come from strictly earlier batches; (D) aggregate counters match the per-problem log exactly. All checks pass on the actual run artifacts. Gold labels exist only inside the 0/1 oracle; the stored hint is always the model's own earlier oracle-passing output, never a dataset field.

Example trace. For a streamed problem, the student prompt contains the system prompt, three retrieved (question, verified-answer) pairs from earlier problems, and the new question—nothing else. The teacher prompt is the identical message list plus one assistant turn containing the model's own stored answer and the question repeated. Full reconstructed traces (built by the training code itself from run artifacts) are released alongside the code.

B Hyperparameters

Base model	Qwen2.5-Coder-7B-Instruct (bf16)
Adapter	LoRA r=16, α=32, dropout 0, all linear layers
Optimizer	AdamW, lr 5×10^{-5}, constant schedule, grad-norm clip 1.0
Distillation	forward KL (α=0), β=0, skip first 3 tokens, temp 1.0
Inner epochs / chunk	2 / grad-accum $\leq$ 50 pairs
Stream	batch 10, window W=9, seed 42
Retrieval	Qwen3-Embedding-0.6B, cosine, k=3, correct-only memory
Decoding	greedy, max 512–1024 new tokens, max sequence 4096
Teacher	frozen base model (never synced)
REINFORCE++	lr 5×10^{-5}, β_{KL}=0.01, PPO clip, whitened adv.
Hardware	1× NVIDIA H200 per run

Table 6: Hyperparameters shared across all experiments.

C Cost Accounting

Per problem, our method spends 1 scored oracle call plus ∼8.8 re-evaluation calls (window re-answers whose only effect is memory update); measured on the fused run: 24,450 total calls for 2,490 problems at the point of measurement (∼9.8/problem). Base, ICL, and REINFORCE++ use exactly 1 call per problem; batch SDFT uses 1 per problem per epoch. Wall-clock on one H200: standalone streams ∼8–14 h; the fused 4,219-problem run ∼30 h of compute (checkpoint-resumable in preemptible 4 h slices); batch SDFT (4 epochs) ∼3.5 h; a bare-prompt evaluation pass ∼40 min. [TODO: Finalize GPU-hour table per run.]

D Additional Ablation Detail

Generation source. DS-1000 standalone, full 955: bare-prompt generation (offline-SDFT default) .436 acc / 539 regret vs hint-conditioned .423 / 551; the gap accumulates in the last third of the stream. Fused stream (stopped at 1,820 of 4,219 by design): bare-prompt loses ∼10 regret in the first 300 problems, then tracks even for 1,500 problems.

Epochs and learning rate. DS-1000, matched settings: 4 epochs −21 cumulative regret vs 2 epochs; 2 epochs at doubled lr +5 regret vs 2 epochs—epochs do what lr cannot.

Multi-sample distillation. N=10 teacher-conditioned samples per problem, similarity-filtered (sim $\geq$ 0.5 to hint) or unfiltered: both statistically indistinguishable from N=1

on DS-1000 (filtered: 186 vs 188 regret at 300 problems; unfiltered: mildly negative over 520 problems).

REINFORCE++ collapse diagnostics. Batch-level metrics around the collapse (fused stream, batch ∼205–210): approximate KL 0.22 → −0.63, PPO clip fraction 0.08 → 0.68, advantage std 0.38 → 1.37 within 10 batches; running accuracy .503 → .253 by stream end. [TODO: Add full trajectory figures for the collapse diagnostics.]

E Per-Task Prompts

[TODO: Verbatim system prompts and generation templates for all seven tasks, and the student/teacher message structure (available in the released repository; port into figures here).]